MINI PROJECT REPORT

STUDENT RECORD SEARCH SYSTEM USING TREES

Submitted By: Dipanshu Sharma
Roll Number: 2556451
Course: BTech CSE A
Subject: Data Structures and Algorithms

1. INTRODUCTION

The Student Record Search System is a mini project based on Trees in Data Structures and Algorithms. It stores student records in a Binary Search Tree (BST), using enrolment ID as the unique key. It supports insertion, duplicate rejection, searching, deletion (all three cases), and Inorder, Preorder, and Postorder traversals. Inorder traversal displays records in sorted enrolment-ID order.

2. OBJECTIVES

• Understand BST ordering properties.
• Store student records using enrolment ID as the key.
• Implement insertion, search, and deletion, rejecting duplicates.
• Handle leaf, one-child, and two-child deletion cases.
• Compare balanced and skewed trees.
• Explain average O(log n), worst-case O(n), and AVL self-balancing trees.

3. BST DATA STRUCTURE DESIGN

Each node stores the following fields:

Field	Type	Purpose
enrolmentID	int	BST search key
name	char array (50)	Student full name
department	char array (50)	Department / programme
left / right	pointer	Pointers to subtrees

4. BST ALGORITHMS & COMPLEXITY ANALYSIS

Deletion cases:
• **Case 1 (Leaf):** Remove the node and set the parent link to NULL.
• **Case 2 (One child):** Replace the node with its child.
• **Case 3 (Two children):** Copy the inorder successor (smallest node in the right subtree), then delete that successor.

5. TIME & SPACE COMPLEXITY COMPARISON

Operation	Average (Balanced)	Worst (Skewed)	AVL Tree
Insertion	O(log n)	O(n)	O(log n)
Search	O(log n)	O(n)	O(log n)
Deletion	O(log n)	O(n)	O(log n)
Tree height	O(log n)	O(n)	O(log n)

6. TEST CASES MATRIX

Test	Description	Input	Expected behaviour	Result
1	Duplicate key	Insert ID 2556451 twice	Duplicate warning	PASS
2	Delete leaf	Delete ID with 0 children	Node deleted cleanly	PASS
3	Delete 1-child node	Delete ID with 1 child	Child promoted	PASS
4	Delete 2-child node	Delete root ID	Replaced by inorder successor	PASS

7. COMPLETE C SOURCE CODE IMPLEMENTATION

C SOURCE CODE — STUDENT RECORD SEARCH SYSTEM

Formatted C source code. Statements and blocks are separated onto readable lines.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct StudentNode {
    int enrolmentID;
    char name[50];
    char department[50];
    struct StudentNode *left;
    struct StudentNode *right;
} StudentNode;

StudentNode *createNode(int id, const char name[], const char dept[]) {
    StudentNode *node = malloc(sizeof(StudentNode));
    if (node == NULL) {
        fprintf(stderr, "Memory allocation failed.\\n");
        exit(EXIT_FAILURE);
    }
    node->enrolmentID = id;
    snprintf(node->name, sizeof(node->name), "%s", name);
    snprintf(node->department, sizeof(node->department), "%s", dept);
    node->left = NULL;
    node->right = NULL;
    return node;
}

StudentNode *insert(StudentNode *root, int id,
                    const char name[], const char dept[]) {
    if (root == NULL)
        return createNode(id, name, dept);

    if (id < root->enrolmentID)
        root->left = insert(root->left, id, name, dept);
    else if (id > root->enrolmentID)
        root->right = insert(root->right, id, name, dept);
    else
        printf("Duplicate rejected! Enrolment ID %d already exists.\\n", id);

    return root;
}

StudentNode *findMin(StudentNode *node) {
    while (node != NULL && node->left != NULL)
        node = node->left;
    return node;
}

StudentNode *deleteNode(StudentNode *root, int id) {
    if (root == NULL)
        return NULL;

    if (id < root->enrolmentID)
        root->left = deleteNode(root->left, id);
    else if (id > root->enrolmentID)
        root->right = deleteNode(root->right, id);
    else {
        /* Case 1 and 2: zero or one child. */
        if (root->left == NULL) {
            StudentNode *temp = root->right;
            free(root);
```

C SOURCE CODE — STUDENT RECORD SEARCH SYSTEM (continued)

```
            return temp;
        }
        if (root->right == NULL) {
            StudentNode *temp = root->left;
            free(root);
            return temp;
        }

        /* Case 3: two children; use the inorder successor. */
        StudentNode *temp = findMin(root->right);
        root->enrolmentID = temp->enrolmentID;
        strcpy(root->name, temp->name);
        strcpy(root->department, temp->department);
        root->right = deleteNode(root->right, temp->enrolmentID);
    }
    return root;
}

void inorder(StudentNode *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("ID: %d | Name: %s | Dept: %s\\n",
               root->enrolmentID, root->name, root->department);
        inorder(root->right);
    }
}

void preorder(StudentNode *root) {
    if (root != NULL) {
        printf("ID: %d | Name: %s\\n", root->enrolmentID, root->name);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(StudentNode *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("ID: %d | Name: %s\\n", root->enrolmentID, root->name);
    }
}

StudentNode *search(StudentNode *root, int id) {
    if (root == NULL || root->enrolmentID == id)
        return root;
    if (id < root->enrolmentID)
        return search(root->left, id);
    return search(root->right, id);
}

void freeTree(StudentNode *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main(void) {
```

C SOURCE CODE — STUDENT RECORD SEARCH SYSTEM (continued)

```
StudentNode *root = NULL;
int choice, id;
char name[50], department[50];

do {
    printf("\\n=== STUDENT RECORD BST SYSTEM ===\\n");
    printf("1. Insert Student\\n2. Search Student\\n3. Delete Student\\n");
    printf("4. Inorder\\n5. Preorder\\n6. Postorder\\n0. Exit\\n");
    printf("Enter choice: ");

    if (scanf("%d", &choice) != 1)
        break;

    switch (choice) {
        case 1:
            printf("Enter enrolment ID, name and department "
                   "(single-word name/department): ");
            if (scanf("%d %49s %49s", &id, name, department) != 3) {
                printf("Invalid student details.\\n");
                break;
            }
            root = insert(root, id, name, department);
            break;
        case 2: {
            printf("Enter ID to search: ");
            if (scanf("%d", &id) != 1)
                break;
            StudentNode *result = search(root, id);
            if (result != NULL)
                printf("Found! Name: %s | Dept: %s\\n",
                       result->name, result->department);
            else
                printf("Student ID %d not found.\\n", id);
            break;
        }
        case 3:
            printf("Enter ID to delete: ");
            if (scanf("%d", &id) != 1)
                break;
            if (search(root, id) == NULL)
                printf("Student ID %d not found.\\n", id);
            else {
                root = deleteNode(root, id);
                printf("Student record deleted successfully.\\n");
            }
            break;
        case 4:
            printf("--- INORDER TRAVERSAL (SORTED) ---\\n");
            inorder(root);
            break;
        case 5:
            printf("--- PREORDER TRAVERSAL ---\\n");
            preorder(root);
            break;
        case 6:
            printf("--- POSTORDER TRAVERSAL ---\\n");
            postorder(root);
            break;
        case 0:
            printf("Exiting Student Record Search System.\\n");
```

C SOURCE CODE — STUDENT RECORD SEARCH SYSTEM (continued)

```
                break;
            default:
                printf("Invalid menu choice.\\n");
        }
    } while (choice != 0);

    freeTree(root);
    return 0;
}
```

8. ASSESSMENT RUBRIC (20 MARKS)

Criteria	Maximum Marks
Data Structure Design & Representation	4
Core Algorithms & Operations (BST)	4
Implementation & Working Output	4
Project Report & Screenshots	2
GitHub Submission	1
Viva / Project Presentation	5